hinglishParser.ts

frontend/utils | TypeScript

```
/**
* hinglishParser.ts
*
* Classifies code-switched Hinglish speech into structured intent/stress signals.
*
* Standard NLP models are trained predominantly on formal English. Real users
* under stress — especially in India — communicate in code-switched Hinglish:
* mixing Hindi vocabulary, grammar, and idiom with English in the same sentence.
*
* "Yaar, deadline sar par hai" would produce near-zero signal in GPT-based
* classifiers trained on English corpora. This parser handles it correctly.
*
* Architecture:
* - Tier 1: Rule-based pattern matching (fast, no API, handles known phrases)
* - Tier 2: Fallback to Groq for ambiguous or novel expressions
* - Output: normalized score 0–100 for the semantic fusion channel (weight: 0.2)
*/
// ■■■ Types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export type IntentClass =
| 'URGENT_DEADLINE'
| 'HIGH_STRESS'
| 'OVERWHELMED'
| 'FOCUS_REQUEST'
| 'NEUTRAL'
| 'POSITIVE';
export type LanguageTag = 'hindi' | 'english' | 'hinglish';
export interface IntentResult {
intent: IntentClass;
score: number; // 0–1 raw confidence
normalized: number; // 0–100 for fusion model
language: LanguageTag;
keywords: string[];
}
// ■■■ Pattern definitions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
//
// Each pattern is (regex, intent, weight).
// Weight is the normalized score (0–1) assigned when this pattern matches.
// Multiple patterns can match; the highest weight wins.
interface Pattern {
regex: RegExp;
intent: IntentClass;
weight: number;
}
const PATTERNS: Pattern[] = [
// ■■ Critical urgency ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{ regex: /deadline\s*(sar|sir|par|aa|aane|kal)\s*(par|hai|raha|wali)/i, intent: 'URGENT_DEADLINE',
weight: 0.95 },
{ regex: /deadline\s*(aa\s*gayi|tonight|today|abhi|urgent)/i, intent: 'URGENT_DEADLINE', weight:
0.90 },
{ regex: /submit\s*(karna|karo|krna)\s*(hai|padega)/i, intent: 'URGENT_DEADLINE', weight: 0.88 },
{ regex: /kal\s*tak\s*(dena|submit|finish|complete)/i, intent: 'URGENT_DEADLINE', weight: 0.85 },
// ■■ High stress ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{ regex: /bahut\s*(zyada|saara|bada)\s*(tension|stress|pressure)/i, intent: 'HIGH_STRESS', weight:
0.90 },
{ regex: /(tension|stress|pareshaan|worried|anxious)\s*(ho\s*raha|hai|hoon)/i, intent: '
HIGH_STRESS', weight: 0.82 },
{ regex: /arre\s*yaar.*?(tension|stress|bura|galat)/i, intent: 'HIGH_STRESS', weight: 0.78 },
{ regex: /kuch\s*samajh\s*nahi\s*(aa|raha)/i, intent: 'HIGH_STRESS', weight: 0.75 },
```

```
{ regex: /dimag\s*(kharab|blast|jam)\s*(ho|gaya|raha)/i, intent: 'HIGH_STRESS', weight: 0.85 },
// ■■ Overwhelmed ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{ regex: /bahut\s*(zyada|saara)\s*(kaam|work|cheez)/i, intent: 'OVERWHELMED', weight: 0.85 },
{ regex: /kuch\s*khatam\s*(hi\s*nahi|nahi)\s*(ho\s*raha|hota)/i, intent: 'OVERWHELMED', weight: 0.80 },
{ regex: /(totally\s*cooked|completely\s*cooked|burnt\s*out)/i, intent: 'OVERWHELMED', weight: 0.88 },
{ regex: /itna\s*(kaam|pressure|sab\s*kuch)\s*(ho\s*raha|hai)/i, intent: 'OVERWHELMED', weight: 0.82 },
{ regex: /overwhelm/i, intent: 'OVERWHELMED', weight: 0.80 },
// ■■ Focus requests ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{ regex: /(dhyan|focus|concentrate)\s*(karna|dena|chahiye)/i, intent: 'FOCUS_REQUEST', weight: 0.70 },
{ regex: /kaam\s*(pe|par)\s*(dhyan|focus)\s*(karo|lagao)/i, intent: 'FOCUS_REQUEST', weight: 0.68 },
// ■■ Positive / neutral ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
{ regex: /(sab\s*)?(theek|sahi|accha|fine|okay|good)\s*(hai|hoon|chal)/i, intent: 'POSITIVE', weight: 0.30 },
{ regex: /(ho\s*gaya|khatam|done|finished|complete)/i, intent: 'POSITIVE', weight: 0.25 },
];
// Hindi vocabulary markers — used for language detection
const HINDI_WORDS = /\b(hai|hoon|kaam|yaar|bhai|bahut|zyada|aaj|kal|nahi|karo|karna|raha|rahi|gaya|gayi|par|pe|se|ko|ka|ki|ke|m...
// ■■■ Public API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
/**
* Parse a Hinglish/English string and return a structured intent result.
*
* @param text - Transcribed speech or typed input (any language)
* @returns IntentResult with normalized score for fusion model
*/
export function parseHinglishIntent(text: string): IntentResult {
if (!text || text.trim().length === 0) {
return { intent: 'NEUTRAL', score: 0.5, normalized: 50, language: 'english', keywords: [] };
}
const normalizedText = text.toLowerCase().trim();
const language = detectLanguage(normalizedText);
// Run all patterns, collect matches
const matches = PATTERNS
.map(p => {
const match = normalizedText.match(p.regex);
return match ? { pattern: p, keywords: match.slice(0).map(k => k.toLowerCase()) } : null;
})
.filter(Boolean) as Array<{ pattern: Pattern; keywords: string[] }>;
if (matches.length === 0) {
return { intent: 'NEUTRAL', score: 0.5, normalized: 50, language, keywords: [] };
}
// Select the match with highest weight
matches.sort((a, b) => b.pattern.weight - a.pattern.weight);
const best = matches[0];
return {
intent: best.pattern.intent,
score: best.pattern.weight,
normalized: Math.round(best.pattern.weight * 100),
language,
keywords: [...new Set(best.keywords)].filter(k => k.length > 2),
};
}
/**
* Detect the dominant language register of the input text.
*/
function detectLanguage(text: string): LanguageTag {
const hindiMatches = text.match(HINDI_WORDS) ?? [];
const totalWords = text.split(/\s+/).filter(w => w.length > 1).length;
if (totalWords === 0) return 'english';
```

```
const hindiRatio = hindiMatches.length / totalWords;
if (hindiRatio >= 0.5) return 'hindi';
if (hindiRatio >= 0.15) return 'hinglish';
return 'english';
}
```